



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

**Mexi Banco: de un monolito a servicios
que se encuentran solos**

Bitácora del laboratorio S07

Alumno: Christian Franco

Cuenta: 117003136

Materia: Sistemas Distribuidos

Grupo: Grupo 2

Fecha: 10 de octubre de 2026

Laboratorio realizado el sábado 3 de octubre de 2026

Introducción

En la sesión anterior (S06) se partió Mexi Banco, hasta entonces un monolito con una sola base de datos, en cinco servicios independientes: **cuenta**, **movimiento**, **transferencia**, **notificacion** y **spei**. Cada uno corre en su propio contenedor y tiene su propia base, pero esta separación introdujo tres problemas que no existían antes:

- (1) ¿Cómo sabe un servicio dónde encontrar a otro, sin que un humano escriba direcciones IP a mano?
- (2) ¿Cómo se evita que los clientes tengan que conocer cinco puertas distintas y puedan invocar operaciones internas?
- (3) ¿Cómo se reparte la carga cuando una sola copia de un servicio no es suficiente?

Este laboratorio resuelve esos tres problemas con tres piezas: un **directorio de servicios** (Eureka), un **API Gateway** y el **balanceo de carga con réplicas**. La última parte del laboratorio experimenta con la caída de una réplica para medir cuánto tarda el sistema en reaccionar.

Nota sobre el entorno: Todos los comandos se ejecutaron en una máquina con Ubuntu, Docker Engine y Docker Compose v2. El proyecto se trabajó sobre la rama `trabajo-s07` del repositorio `mexi-banco`, derivada de la rama base `lab/03-gateway-discovery-balanceo`.

1. La historia: partimos el banco

Contexto

En el monolito original, una transferencia era una llamada a método dentro del mismo proceso y la misma base de datos:

```
@Transactional
public Transferencia transferir(String origen, String destino, BigDecimal monto)
{
    Cuenta o = cuentas.cargar(origen, monto);
    Cuenta d = cuentas.abonar(destino, monto);
    ...
}
```

Gracias a la anotación `@Transactional`, si el abono fallaba, la base deshacía también el cargo. El sistema garantizaba la atomicidad de la operación.

Al separar en servicios, el mismo método ahora hace peticiones HTTP a otro proceso con su propia base:

```
public Transferencia transferir(String origen, String destino, BigDecimal monto)
{
    CuentaRemota o = cuentas.cargar(origen, monto);
    CuentaRemota d = cuentas.abonar(destino, monto);
    ...
}
```

Qué se rompió

Ya no existe una transacción compartida entre **transferencia** y **cuenta**. Si el abono falla, el cargo ya quedó aplicado y no se deshace automáticamente. Se ganó independencia de despliegue, pero se perdió atomicidad: cada servicio confirma solamente su parte.

Consecuencia para la S08

Este hueco de atomicidad es exactamente lo que se aborda en la siguiente sesión mediante sagas y compensaciones. El laboratorio lo anticipa deliberadamente.

2. El directorio: que cada servicio se anote solo

2.1 Qué es un directorio de servicios

Un *service discovery* es una agenda viva que registra qué servicios existen y en qué dirección está cada instancia. Funciona así:

1. Cada servicio, al arrancar, se anota con su nombre lógico (**cuenta**, **movimiento**, ...) y su IP.
2. Envía un *heartbeat* cada cierto intervalo para indicar que sigue vivo.
3. Si el directorio deja de recibir heartbeats durante un plazo configurado, elimina al servicio de la agenda.
4. Quien quiera hablar con **cuenta** le pregunta al directorio qué instancias hay vivas en ese momento.

Se utilizó **Eureka** (Spring Cloud Netflix) con los siguientes parámetros, tomados de `x-directorio` en el `docker-compose.yml`:

Parámetro	Valor
Heartbeat (EUREKA_LATIDO_S)	5 s
Expiración (EUREKA_EXPIRA_S)	15 s
Consulta del cliente (EUREKA_CONSULTA_S)	5 s

2.2 Cambios realizados

1. En `discovery/.../DiscoveryApplication.java` se agregaron el `import org.springframework.cloud.net` y la anotación `@EnableEurekaServer` sobre la clase. Sin ella, la aplicación no arranca correctamente porque el cliente Eureka incluido en el *starter* busca un servidor que no existe.
2. En `discovery/src/main/resources/application.yml` se desactivó el autorregistro y la consulta de la lista a otros directorios:

```
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

- En cada uno de los cinco `pom.xml` (cuenta, movimiento, transferencia, notificacion, spei) se descomentó la dependencia del cliente de Eureka:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

- En `docker-compose.yml` se descomentaron el servicio `discovery` completo y las dos líneas de `depends_on` en el bloque común `x-servicio`, para que los servicios esperen al directorio antes de arrancar.

2.3 Resultados observados

Antes de los cambios, la consola del navegador en `http://localhost:8761` no cargaba ningún contenido. `http://localhost:8081/cuentas/000` sí respondía directamente a `cuenta` con un 404 en formato *Problem Details*.

Después de reconstruir con `docker compose up -d --build` (los siete contenedores quedaron en estado `healthy`), el tablero de Eureka mostró los cinco servicios registrados:

Servicio	Estado
CUENTA	UP (1)
MOVIMIENTO	UP (1)
NOTIFICACION	UP (1)
SPEI	UP (1)
TRANSFERENCIA	UP (1)

El API XML del directorio devolvió, para CUENTA, un documento con `<status>UP</status>`, `<port>8080</port>`, `<renewalIntervalInSecs>5</renewalIntervalInSecs>` y `<durationInSecs>15</durationInSecs>` exactamente los valores configurados.

spring Eureka			
System Status		HOME LAST 1000 SINCE STARTUP	
Environment	test	Current time	2026-10-10T02:14:58+0000
Data center	default	Uptime	00:03
		Lease expiration enabled	true
		Renews threshold	10
		Renews (last min)	120
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.			
DS Replicas			
localhost			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 84445c71140a:cuenta:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 412c9ed83e69:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - b0c4599e716a:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - b5311684920e7:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - ee2020cf70d3:transferencia:8080
General Info			
Name	Value		
total-avail-mem-mv	80mb		

General Info	
Name	Value
total-avail-memory	82mb
num-of-cpus	8
current-memory-usage	67mb (81%)
server-up-time	00:03
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/
available-replicas	
Instance Info	
Name	Value
ipAddr	172.24.0.3
status	UP

Respuesta al apartado “Piensa”

El directorio ya conoce a los cinco servicios, pero nadie lo está usando todavía para resolver nombres: **transferencia** sigue llamando a `http://cuenta:8080` mediante una URL escrita a mano. El directorio contiene la información, pero aún no hay un componente que la consulte para enrutar peticiones. Ese componente es el *gateway*, del siguiente paso.

3. El gateway: una sola puerta

3.1 Problema que resuelve

Antes del gateway, cada servicio exponía su propio puerto (8081 a 8085) y cualquiera podía invocar operaciones que deberían ser internas. La siguiente petición, lanzada desde la máquina del alumno, lo demuestra:

```
$ curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
{"title":"Bad Request","status":400,"detail":"Required parameter 'clabe' is not present."}
```

Un 400 (y no un 404) significa que el endpoint existe y está abierto al exterior.

El **API Gateway** resuelve ambos problemas: es la única puerta de entrada (puerto 8080) y aplica reglas de enrutamiento. Lo que no tiene regla no pasa.

3.2 Rutas configuradas

En `gateway/src/main/resources/application.yml` se definieron cinco rutas. La sintaxis `lb://nombre` indica al gateway que debe preguntarle al directorio por las instancias de ese servicio y repartir entre ellas.

Ruta	Método	Destino
/api/cuentas, /api/cuentas/{clabe}	GET, POST	lb://cuenta
/api/movimientos	GET	lb://movimiento
/api/transferencias, /api/transferencias/{id}	GET, POST	lb://transferencia
/api/notificaciones	GET	lb://notificacion
/api/spei	POST	lb://spei

El filtro `StripPrefix=1` elimina el segmento `/api` antes de reenviar la petición. Así, `/api/cuentas/123` llega a cuenta como `/cuentas/123`.

3.3 Pruebas

Se realizaron tres pruebas. Las dos primeras verifican quién atiende la petición; la tercera verifica el flujo completo desde el cliente.

Prueba A. Cuenta inexistente por el gateway.

```
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
X-Instancia: 94445c71140a
{"title":"Not Found","status":404,"detail":"No existe la CLABE 000"}
```

Trae la cabecera `X-Instancia`: la petición atravesó el gateway y llegó a cuenta, que fue quien respondió.

Prueba B. POST a movimientos por el gateway.

```
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
{"error":"Not Found","requestId":"550af9fd-94"}
```

No trae `X-Instancia`. El JSON lo generó el propio gateway, que no tiene ruta para POST a `/api/movimientos`. La operación interna quedó blindada.

Prueba C. Alta y consulta de una cuenta.

Se creó una cuenta con una CLABE generada a partir del `timestamp`, y se consultó:

```
$ curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"S071791601064","titular":"S07","saldoInicial":0}'
HTTP/1.1 201 Created

$ curl -i http://localhost:8080/api/cuentas/S071791601064
HTTP/1.1 200 OK
X-Instancia: 94445c71140a
{"id":1,"clabe":"S071791601064","titular":"S07","saldo":0.00}
```

El alta devolvió 201 y la consulta 200, con la cabecera `X-Instancia` que confirma que la petición pasó por el gateway hasta cuenta.

```
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
> curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 94445c71140a
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 02:56:45 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
> █
```

```
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
> curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-10T02:57:20.117+00:00","path":"/api/movimientos","status":404,"error":"Not Found","request
Id":"01b5d6bb-56"}
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
>
Id":"01b5d6bb-56"}
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
> LAB_CLABE="S07$(date +%s)"
echo "Tu CLABE es: $LAB_CLABE"
Tu CLABE es: S071791601064

christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
> curl -i -X POST http://localhost:8080/api/cuentas \
-H 'Content-Type: application/json' \
-d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 94445c71140a
Content-Type: application/json
Date: Sat, 10 Oct 2026 02:57:49 GMT

{"id":1,"clabe":"S071791601064","titular":"S07","saldo":0}
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
>
```

4. Tres copias de cuenta

4.1 Cambio en el compose

En `docker-compose.yml` se sustituyeron las dos líneas de `ports` del servicio `cuenta` por un bloque `deploy` con tres réplicas:

```
cuenta:
  <<: *servicio
  build: ./cuenta
  deploy:
    replicas: 3
  environment:
    <<: *entorno-base
    DB_NAME: cuenta
    MOVIMIENTO_URL: http://movimiento:8080
```

Por la misma razón se eliminaron los `ports` de `movimiento`, `transferencia`, `notificacion` y `spei`. Solo quedaron publicados los puertos del gateway (8080) y del directorio (8761). Los servicios internos siguen escuchando en su 8080 dentro de la red de Docker, pero ya no se alcanzan desde la máquina del alumno.

4.2 Verificación

Tras aplicar el cambio con `docker compose up -d`, se listaron los contenedores de cuenta:

```
$ docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}'
5dddc72e486c mexi-banco-v06-cuenta-1 8080/tcp
75d5bcc1fa18 mexi-banco-v06-cuenta-3 8080/tcp
04bef2f5e1e3 mexi-banco-v06-cuenta-2 8080/tcp
```

Los tres escuchan en 8080/tcp sin flecha, es decir, sin puerto publicado en la máquina anfitriona. En el tablero de Eureka, CUENTA pasó a mostrarse como **UP (3)**.

4.3 Round robin

Se consultó nueve veces consecutivas la misma cuenta a través del gateway:

```
$ for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: 75d5bcc1fa18
X-Instancia: 5dddc72e486c
X-Instancia: 04bef2f5e1e3
X-Instancia: 75d5bcc1fa18
X-Instancia: 5dddc72e486c
X-Instancia: 04bef2f5e1e3
X-Instancia: 75d5bcc1fa18
X-Instancia: 5dddc72e486c
X-Instancia: 04bef2f5e1e3
```

Los tres identificadores se alternaron de manera cíclica sin repetirse dos veces seguidas, confirmando el reparto *round robin* del balanceador del gateway.

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 5dddc72e486c:cuenta:8080 , 75d5bcc1fa18:cuenta:8080 , 04bef2f5e1e3:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - d7daa7c9b44a:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 2304cbdd192d:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 534edef0d560:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - ba82ad539ac2:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 12aaebb2c37b:transferencia:8080
General Info			

```
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
> for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: 75d5bcc1fa18
X-Instancia: 5dddc72e486c
X-Instancia: 04bef2f5e1e3
X-Instancia: 75d5bcc1fa18
X-Instancia: 5dddc72e486c
X-Instancia: 04bef2f5e1e3
X-Instancia: 75d5bcc1fa18
X-Instancia: 5dddc72e486c
X-Instancia: 04bef2f5e1e3
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 !?
> █
```

Respuesta al apartado “Piensa”

El gateway sí reparte. Sin embargo, cuando **transferencia** llama a **cuenta** por dentro, no lo hace: sigue usando **CUENTA_URL**: `http://cuenta:8080`, una dirección fija que el DNS de Docker resuelve a una sola instancia. **Transferencia** no se entera de que hay tres copias. Esto se corrige en la tarea.

5. ¿Y si una copia se muere?

5.1 Predicción previa

Antes de ejecutar el experimento se anotó la siguiente predicción:

Predicción

Van a fallar **algunas** consultas (aproximadamente una de cada tres, las que por turno caen en la copia muerta), durante unos **20 a 30 segundos**, y dejarán de fallar **después** de que la copia desaparezca del directorio, porque el gateway conserva una caché de la lista que refresca periódicamente.

5.2 Procedimiento

Se utilizaron dos terminales. En la primera se lanzó un ciclo de 60 consultas separadas por un segundo, imprimiendo hora y código HTTP:

```
for i in {1..60}; do
  echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
  sleep 1
done
```

En la segunda terminal se registró la hora del apagado y se mató una copia con `docker kill` (corte abrupto, sin `stop`):

```
$ date +%T
21:15:44
$ docker kill mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

Mientras tanto, se refrescó periódicamente el tablero de Eureka hasta observar que CUENTA pasara de UP (3) a UP (2).

5.3 Resultados

El ciclo produjo 60 líneas. Se observaron 6 respuestas 500 en la ventana comprendida entre las 21:15:44 y las 21:16:08, es decir, aproximadamente 24 segundos de errores intermitentes:

```
21:15:44 500
21:15:47 200
21:15:49 500
21:15:53 500
21:15:59 500
21:16:04 500
21:16:08 500
21:16:11 200
...
```

Antes de las 21:15:44 todas las respuestas fueron 200; a partir de las 21:16:11 también. Las seis fallas fueron 500, no 503, porque el gateway intentó contactar a la réplica muerta, recibió un error de conexión y lo propagó como error interno.

```

done
21:15:15 200
21:15:16 200
21:15:17 200
21:15:18 200
21:15:19 200
21:15:20 200
21:15:21 200
21:15:22 200
21:15:23 200
21:15:24 200
21:15:25 200
21:15:26 200
21:15:27 200
21:15:28 200
21:15:29 200
21:15:30 200
21:15:31 200
21:15:32 200
21:15:33 200
21:15:34 200
21:15:35 200
21:15:36 200
21:15:37 200
21:15:39 200
21:15:40 200
21:15:41 200
21:15:42 200
21:15:43 200
21:15:44 500
21:15:47 200
21:15:48 200
21:15:49 500
21:15:52 200
21:15:53 500
21:15:56 200
21:15:57 200
21:15:58 200
21:15:59 500
21:16:02 200
21:16:03 200
21:16:04 500
21:16:07 200
21:16:08 500
21:16:11 200
21:16:12 200
21:16:13 200
21:16:14 200
21:16:15 200
21:16:16 200
21:16:17 200
21:16:18 200
21:16:19 200
21:16:20 200
21:16:21 200
21:16:22 200
21:16:23 200
21:16:24 200
21:16:25 200
21:16:26 200
21:16:27 200

```

La réplica salió del directorio aproximadamente a los 20 segundos del apagado, consistente con los 15 s de plazo (`durationInSecs`) más el ciclo de barrido de 5 s (`eviction-interval-timer-in-ms`).

5.4 Explicación del retardo

El sistema no detecta la caída al instante porque nadie avisa: la información fluye por capas.

Capa	Cómo se entera	Tiempo
Directorio (Eureka)	Deja de recibir heartbeats y espera el plazo antes de borrar	15 s + 5 s de barrido
Gateway	Refresca la copia que tiene de la lista del directorio	cada 5 s
Balanceador	Caché de instancias	5 s
Total	Ventana de errores intermitentes	≈ 25–30 s

Con los valores por defecto de Spring (latido 30 s, expiración 90 s) la ventana sería de varios minutos. Reducir los tiempos detecta antes pero incrementa la carga sobre el directorio.

5.5 Recuperación

Se revivió la réplica con `docker start mexi-banco-v06-cuenta-1`. Tras aproximadamente 60–90 segundos (arranque de la JVM más registro en Eureka), CUENTA volvió a mostrarse como **UP (3)** en el tablero.

6. Cierre

Síntesis conceptual

- El **directorío** resuelve saber qué servicios existen y en dónde están, sin que nadie escriba direcciones a mano. Cada servicio se anota solo y envía heartbeats.
- El **gateway** resuelve ofrecer una sola puerta de entrada con reglas: lo que no tiene ruta no pasa, y las operaciones internas quedan fuera del alcance del cliente.
- El **balanceador** resuelve repartir carga entre copias idénticas y dejar de enviar peticiones a las que mueren, sin intervención humana.
- Cuando murió una réplica se observaron seis errores 500 intermitentes durante aproximadamente 24 segundos, porque la detección no es instantánea: depende del plazo del directorio, del refresco del gateway y de la caché del balanceador.

Reflexión

Los sistemas distribuidos no fallan de una sola manera: fallan parcialmente y durante un tiempo que depende de los mecanismos de detección configurados. Entender ese intervalo y saber cuantificarlo es tan importante como saber programar los servicios.

7. Tarea: que transferencia también reparta

7.1 Objetivo

Hasta este punto, solo el gateway repartía entre las tres réplicas de **cuenta**. Internamente, **transferencia** seguía llamando a `http://cuenta:8080`, una dirección que el DNS de Docker resuelve a una sola instancia. La tarea consistió en hacer que **transferencia** también consulte al directorio y reparta.

7.2 Cambios realizados

T.1. Bean @LoadBalanced. En `transferencia/src/main/java/mx/mexibanco/clientes/CientesHttp.java` se agregaron dos imports y un método que construye un `RestClient.Builder` balanceado:

```
import org.springframework.boot.autoconfigure.web.client.  
    RestClientBuilderConfigurer;  
import org.springframework.cloud.client.loadbalancer.LoadBalanced;
```

```
@Bean
@LoadBalanced
RestClient.Builder restClientBalanceado(RestClientBuilderConfigurer configurador)
{
    return configurador.configure(RestClient.builder());
}
```

La anotación `@LoadBalanced` le indica a Spring que el host de la URL debe interpretarse como un nombre de servicio y sustituirse, en cada petición, por la IP y el puerto de una instancia registrada en Eureka.

T.2. URLs sin puerto. En `docker-compose.yml`, dentro del servicio `transferencia`, se eliminó el `:8080` de las tres URLs:

```
CUENTA_URL: http://cuenta
MOVIMIENTO_URL: http://movimiento
NOTIFICACION_URL: http://notificacion
```

El servicio `spei` no se tocó; sigue con sus URLs literales.

T.3. Reconstrucción y prueba. Se reconstruyó únicamente el servicio afectado y se ejecutó la demo completa:

```
docker compose up -d --build transferencia
./demo-v06.sh
```

7.3 Resultados

La demo corrió de principio a fin. Los resultados más relevantes fueron:

Verificación	Resultado
Alta de Ana y Beto	201 Created
Transferencia de 200	Saldos finales 800 / 700
SPEI idempotente	Mismo <code>idempotencyKey</code> → mismo <code>id</code> ; saldo de Ana en 750
Round robin por el gateway	Tres IDs alternándose, cero repeticiones seguidas
Endpoints internos por el gateway	404 (<code>/cargos</code> , <code>/abonos</code>)
Errores de negocio	404 (CLABE inexistente) y 422 (saldo insuficiente) preservados con <code>servicio: cuenta</code>
Saldos finales	Ana 750, Beto 700

7.4 Observación relevante: timeout en la primera transferencia

La primera transferencia de la demo devolvió 503 con `Read timed out`, pero los saldos sí cambiaron. Los logs del servicio `cuenta` muestran que `transferencia` hizo todas las llamadas internas:

```
cuenta-2 03:45:47.386 POST /cuentas/...001/cargos atendida por 04bef2f5e1e3
cuenta-3 03:45:47.611 POST /cuentas/...002/abonos atendida por 75d5bcc1fa18
```

El cargo y el abono de la misma transferencia cayeron en réplicas distintas. Esto confirma que `transferencia` ya no depende de una copia específica. El 503 fue un problema de temporización: la

operación se completó, pero la respuesta tardó más de lo que el cliente estaba dispuesto a esperar. Ese comportamiento —el cliente no sabe si la operación se realizó o no— es precisamente el tema que se abordará en la siguiente sesión.

```
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 1?
> ./demo-v06.sh

=== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ===
-> listo en 1 s. Lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA      3 instancia(s)
GATEWAY     1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI        1 instancia(s)
TRANSFERENCIA 1 instancia(s)
-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar

=== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009160394001","titular":"Ana","saldoInicial":1000}'
{
  "id": 2,
  "clabe": "002180009160394001",
  "titular": "Ana",
  "saldo": 1000
}
[HTTP 201]
-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas

=== 2. Abrir la cuenta de Beto con 500 (misma puerta) ===
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009160394002","titular":"Beto","saldoInicial":500}'
{
  "id": 3,
  "clabe": "002180009160394002",
  "titular": "Beto",
  "saldo": 500
}
[HTTP 201]

=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009160394001","claveDestino":"002180009160394002","monto":200}'
{
  "type": "about:blank",
  "title": "Service Unavailable",
  "status": 503,
  "detail": "No se pudo contactar a otro servicio: Could not retrieve response status code: java.net.SocketTimeoutException: Read timed out",
  "instance": "/transferencias"
}
[HTTP 503]
-> transferencia llamo a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
cuenta-3 GET /cuentas/002180009160394001 atendida por 75d5bcc1fa18
cuenta-1 GET /cuentas/002180009160394002 atendida por 5dddc72e486c
cuenta-2 POST /cuentas/002180009160394001/cargos atendida por 04bef2f5e1e3
cuenta-3 POST /cuentas/002180009160394002/abonos atendida por 75d5bcc1fa18
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

=== 4. Consultar esa transferencia ===
$ curl http://localhost:8080/api/transferencias/
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No static resource transferencias.",
  "instance": "/transferencias/"
}
[HTTP 404]

=== 5. Saldos: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009160394001
{
  "id": 2,
  "clabe": "002180009160394001",
  "titular": "Ana",
  "saldo": 800.00
}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009160394002
{
  "id": 3,
  "clabe": "002180009160394002",
  "titular": "Beto",
  "saldo": 700.00
}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700
```

```

=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===
$ curl http://localhost:8080/api/movimientos?clabe=002180009160394001
[
  {
    "id": 3,
    "clabeCuenta": "002180009160394001",
    "tipo": "TRANSFERENCIA_ENVIADA",
    "monto": -200.00,
    "saldoResultante": 800.00,
    "fecha": "2026-10-10T03:45:48.373595Z",
    "referencia": "a 002180009160394002"
  },
  {
    "id": 1,
    "clabeCuenta": "002180009160394001",
    "tipo": "DEPOSITO",
    "monto": 1000.00,
    "saldoResultante": 1000.00,
    "fecha": "2026-10-10T03:45:43.630062Z",
    "referencia": "saldo inicial"
  }
]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clabe=002180009160394002
[]
[HTTP 200]

=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91603940' -d '{"clabeOrigen": "002180009160394001", "bancoDestino": "BANCO-X", "claveDestino": "0121800000000000009", "monto": 50}'
[
  {
    "id": 1,
    "idempotencyKey": "spei-demo-91603940",
    "clabeOrigen": "002180009160394001",
    "bancoDestino": "BANCO-X",
    "claveDestino": "0121800000000000009",
    "monto": 50,
    "estado": "ENVIADO"
  }
]
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91603940' -d '{"clabeOrigen": "002180009160394001", "bancoDestino": "BANCO-X", "claveDestino": "0121800000000000009", "monto": 50}'
[
  {
    "id": 1,
    "idempotencyKey": "spei-demo-91603940",

```

```

{
  "id": 1,
  "idempotencyKey": "spei-demo-91603940",
  "clabeOrigen": "002180009160394001",
  "bancoDestino": "BANCO-X",
  "claveDestino": "0121800000000000009",
  "monto": 50.00,
  "estado": "ENVIADO"
}
[HTTP 201]
-> mismo id (1 -> 1): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)

=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===
consulta 1 X-Instancia: 5dddc72e486c (mexi-banco-v06-cuenta-1)
consulta 2 X-Instancia: 04bef2f5e1e3 (mexi-banco-v06-cuenta-2)
consulta 3 X-Instancia: 75d5bcc1fa18 (mexi-banco-v06-cuenta-3)
consulta 4 X-Instancia: 5dddc72e486c (mexi-banco-v06-cuenta-1)
consulta 5 X-Instancia: 04bef2f5e1e3 (mexi-banco-v06-cuenta-2)
consulta 6 X-Instancia: 75d5bcc1fa18 (mexi-banco-v06-cuenta-3)
consulta 7 X-Instancia: 5dddc72e486c (mexi-banco-v06-cuenta-1)
consulta 8 X-Instancia: 04bef2f5e1e3 (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: 75d5bcc1fa18 (mexi-banco-v06-cuenta-3)
-> 3 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080

=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/002180009160394001/abonos -H 'Content-Type: application/json' -d '{"monto": 1000000}'
{
  "timestamp": "2026-10-10T03:46:05.485+00:00",
  "path": "/api/cuentas/002180009160394001/abonos",
  "status": 404,
  "error": "Not Found",
  "requestId": "8bbd93a7-732"
}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

```

```

=== 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":
:"002180009160394001","claveDestino":"002180009160394099","monto":10}'
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009160394099",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":
:"002180009160394001","claveDestino":"002180009160394002","monto":99999}'
{
  "type": "about:blank",
  "title": "Unprocessable Entity",
  "status": 422,
  "detail": "Saldo insuficiente en la CLABE 002180009160394001",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido

christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 ?

```

```

christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 ?
) docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-3 | 2026-10-10T03:45:47.611Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstan
cia : POST /cuentas/002180009160394002/abonos atendida por 75d5bcc1fa18
cuenta-3 | 2026-10-10T03:46:01.217Z INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstan
cia : POST /cuentas/002180009160394001/cargos atendida por 75d5bcc1fa18
cuenta-2 | 2026-10-10T03:45:47.386Z INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstan
cia : POST /cuentas/002180009160394001/cargos atendida por 04bef2f5e1e3
cuenta-1 | 2026-10-10T03:46:05.549Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstan
cia : POST /cuentas/002180009160394001/cargos atendida por 5dddc72e486c

christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_christian/s07/mexi-banco
on trabajo-s07 ?

```

Conclusión general

Este laboratorio mostró cómo tres piezas —directorío, gateway y balanceador— resuelven problemas que aparecen naturalmente al partir un monolito en servicios. También expuso que la detección de fallas en sistemas distribuidos tiene un costo inevitable en latencia: no existe una respuesta instantánea, solo configuraciones más o menos agresivas que balancean velocidad de detección contra carga sobre el directorío.

El ejercicio final, con el 503 de la primera transferencia, dejó planteado el problema que abre la siguiente sesión: cuando una operación distribuida no puede confirmarse, el sistema necesita mecanismos adicionales para garantizar consistencia.